



최현준

Choi Hyeonjun

Backend Engineer | 사용자 문제를 데이터와 코드로 해결하는 백엔드 개발자

☎ 010-9485-1049 ✉ osumaniaddict527@gmail.com 📅 2000.05.27

🌐 osumaniaddict527.tistory.com 🌐 github.com/Hyeonjun0527 🌐 yeon.world

자기소개

사용자가 겪는 지연·오류를 **로그, DB, 실제 요청·응답으로 확인한 뒤 원인에 맞는 코드와 운영 변경을 적용하는** 백엔드 개발자입니다.
YEON Platform · NEXA · ZERO-ONE 등 여러 서비스를 직접 개발하고 운영하고 있습니다.

학력

강원대학교 컴퓨터공학과 재학 · 2026.09 졸업 예정

2020.03 ~ 2026.09

전체 학점 4.0 / 4.5 · 전공 학점 4.2 / 4.5 · 주요 이수 과목 컴퓨터 구조 · 자료구조 · 알고리즘 · 운영체제

주요 기술

Backend	Java · Kotlin · Spring Boot · Spring MVC · JPA · MyBatis · Spring Security · OAuth2 · REST API
Database / Messaging	MySQL · MariaDB · PostgreSQL · Oracle · Redis · RabbitMQ · Qdrant
Infra / DevOps	Docker · Nginx · AWS · Naver Cloud Platform · GitHub Actions · cloudflared · Linux · CI/CD
Frontend / Tools	React · TypeScript · JavaScript · JSP/JSTL · HTML · CSS · Git · Postman · Jira · Slack · Notion

수상 및 활동

- 🏆 카카오 테크 캠퍼스 최우수상 — 상금 1,000만원
- 📄 네이버 클라우드 부트캠프 수료 (2024.01 ~ 2024.07)
- 🏆 성적 장학금 총 5회
- 📄 산학협력 프로젝트 — Discord AI 멤버 시스템 설계·구현
- 🔗 교내 알고리즘 동아리 Alergy (2025.01 ~ 현재)
- 📄 카카오 테크 캠퍼스 수료 (2025.04 ~ 2025.11)
- 🏆 전액 장학금 1회 — 같은 학번 170명 중 1명 수여
- 🏆 메이저 러너 우수상 — 교내 학습 공동체
- 📄 OSSCA 체험형 과정 수료 (2024.04 ~ 2024.05)

자격증

- 🏆 NAVER Cloud Platform Certified Professional ·
- 2024.07.12

프로젝트 소개

프로젝트	프로젝트 소개	역할	핵심 스택
Dailyting	랜덤 매칭 후 3일간 일상을 공유하고, 마지막 선택으로 관계를 이어가는 소개팅 애플리케이션	백엔드 오너 · 2인 제품팀	Kotlin · Spring Boot · R2 · ffmpeg
NEXA	Discord에서 대화 맥락과 관계를 기억하며 소통하는 AI 멤버 시스템	프로젝트 리드(1인)	Kotlin · Spring Boot · JDA · PostgreSQL
YEON Platform	타자연습·카드학습·백지학습·플래너·게임·커뮤니티·뉴스를 한 브랜드로 운영하는 멀티 서비스 플랫폼	프로젝트 리드(1인)	Spring Boot · Nginx · cloudflared
PULL-IT	PDF를 분석해 AI가 학습 문제를 생성하는 플랫폼	팀장(5인)	Spring Boot · RabbitMQ · SSE
ZERO-ONE	개발자 스터디 매칭·커뮤니티·랭킹을 제공하는 학습 플랫폼	백엔드(8인 팀)	Spring Batch · JPA · MySQL

프로젝트별 문제해결 한 줄 소개

프로젝트	문제해결 한 줄 소개
Dailyting	영상 재생 정보를 파일 앞에 배치해 재생 시작 대기 조건을 개선
NEXA	대화 흐름을 보고 말할지 기다릴지 판단해 불필요한 발화를 제어
YEON Platform	매 요청마다 반복하던 랭킹 전체 집계를 짧은 캐시로 전환
PULL-IT	긴 AI 문제 생성은 Worker로 분리하고 사용자 요청에는 먼저 응답
ZERO-ONE	유사 학습자를 자동 매칭하면서 같은 상대의 반복 가능성을 낮춤

백엔드 핵심 경험

경험 영역	프로젝트 경험
유실과 중복에 민감한 백엔드 설계	PULL-IT에서 요청 서버와 AI 작업 서버를 분리하고, 완료 알림이 실패하면 다시 전달하도록 구현했습니다.
메시지 기반 비동기 처리와 배치 작업	RabbitMQ로 작업을 비동기 처리하고, Spring Batch로 주기적인 매칭 작업을 실행했습니다.
성능 병목 측정과 구조 개선	YEON의 랭킹 기능을 부하 테스트하고, Dailyting의 영상 파일 구조와 PULL-IT의 문제 저장 경로를 변경 전후로 측정했습니다.
외부 API·AI 호출의 실패 격리	PULL-IT의 Gemini Worker 실패를 처리하고, YEON의 Google·Kakao API 호출에 타임아웃과 공용 HTTP 클라이언트를 적용했습니다.
서비스 운영·배포와 장애 대응	Dailyting과 PULL-IT을 운영하며 배포 자동화와 장애 확인 절차를 구성했습니다.

백엔드 기술 경험 요약

프로젝트와 로컬 실험에서 직접 다룬 백엔드 기술과 적용 내용을 정리했습니다.

기술 영역	적용 경험 및 결과
MySQL/PostgreSQL 쿼리 병목 분석	YEON 랭킹이 요청마다 전체 좋아요를 다시 묶고 정렬하는 구조를 DB 실행 계획으로 확인. 기존 집계 46.8ms, 요약 테이블·정렬 인덱스 대안 0.067ms
로컬/인메모리 캐시	YEON에서 결과를 5초 동안 보관하고 변경 시 즉시 비워, 동일 로컬 조건의 p95 151.9ms → 1.4ms
메시지 큐 기반 비동기 처리	PULL-IT에서 요청 서버가 작업 서버에 일을 보내고, 작업 서버가 완료 사실을 다시 보내는 두 방향 큐 설계
트랜잭션/정합성	DB 저장이 끝난 뒤에만 작업 메시지를 보내고, 생성 성공과 알림 실패를 다른 상태로 기록. 문제 저장은 한 트랜잭션으로 묶음
DB 커넥션 풀/타임아웃 설정	서비스별 커넥션 풀·타임아웃 정책과 헬스 체크를 구성하고, 운영값은 공개 문서에서 제외
외부 HTTP 호출 timeout/재사용	YEON의 Google/Kakao 호출에 공용 HTTP 클라이언트, 연결 5초 제한, 요청별 제한 시간을 적용하고 관련 테스트 통과
Java 21 가상스레드	100ms 동안 대기하는 API를 로컬 k6 200 VU·20초로 비교: 일반 스레드 310.8 요청/초·p95 688ms → 가상 스레드 1,912.8 요청/초·p95 107.3ms
tcpdump 패킷 분석	Docker Compose에서 HTTP 98개·AMQP 29개 패킷을 pcap으로 저장하고 TCP 3-way handshake, GET/200, AMQP 헤더를 판독
JVM/JFR 관찰	k6 80 VU·20초에서 HikariCP 8개가 모두 점유되고 최대 42개 스레드가 대기. JFR의 GC 19회·ThreadPark 4,997건과 jcmd의 커넥션 대기 스택을 확인
Tomcat·Netty 스레드 모델 비교	100ms 대기 API를 k6 300 VU·20초로 비교: Tomcat 1,946.1 req/s·p95 189.2ms·요청 스레드 200개, Netty 2,948.2 req/s·p95 104.3ms·이벤트 루프 4개
HTTP host별 연결·요청 제한	200개 요청에서 Apache/OkHttp의 host별 제한을 5→20으로 변경하자 처리량 약 4배, p95 약 74% 감소. JDK 기준값을 포함한 모든 시나리오 오류 0건

성능 수치는 표에 적힌 동일 로컬 조건의 비교 결과이며, 운영 환경의 처리량을 의미하지 않습니다.

Dailyting — MP4 파일 구조를 변경해 영상 재생 시작 지연 개선

미디어 처리

백엔드 오너

2026.05 ~ 현재 [dailyting.cloud](#)

Kotlin

Spring Boot

Cloudflare R2

AWS SDK v2

ffmpeg

KMP/Compose

운영 범위

운영 도메인 16개

공개 제품 URL과 안전한 공개 기술 문서만 제공합니다.

Dailyting은 정적 썸네일 대신 영상을 무음으로 반복 재생해 미리보기를 제공합니다. 영상은 정상적으로 불러와졌지만 재생 시작이 늦어, 첫 화면이 한동안 표시되지 않았습니다. 업로드 기록과 R2 객체, HTTP 응답을 확인한 뒤 원본 MP4 파일을 분석한 결과, 재생에 필요한 **moov atom이 파일 끝부분에 배치**돼 있었습니다. 이에 업로드 완료 요청에서 FFmpeg의 faststart 처리를 동기적으로 수행해 moov atom을 파일 앞부분으로 이동했습니다.

문제 · 확인 · 결과

문제	영상 미리보기 재생 시작이 늦어 사용자가 첫 화면을 늦게 확인함 (썸네일 대신 무음 영상 재생 사용)
확인	DB·R2·API 응답은 정상, 실제 MP4는 ftyp → mdat → moov 순서로 재생 정보가 파일 끝에 배치됨
구조 측정	5초 테스트 영상에서 파일 시작~moov atom 끝 범위가 약 829KB → 4.4KB. 단말 첫 프레임 시간은 별도 측정 대상

문제

- 영상 미리보기의 재생 시작이 늦어 사용자가 첫 화면을 확인하기까지 지연이 발생했습니다.

확인한 사실

- 업로드 기록, R2 객체, HTTP 응답에는 이상이 없었습니다.
- MP4 파일이 **ftyp → mdat → moov** 순서로 구성돼 있었습니다.
- 재생 정보가 담긴 **moov** atom이 파일 끝부분에 있어 플레이어가 재생 정보를 늦게 확인하고 있었습니다.

바꾼 점

- 영상 업로드 완료 요청에서 FFmpeg의 faststart 처리를 동기적으로 수행했습니다: **ffmpeg -i input.mp4 -c copy -movflags +faststart output.mp4**
- 영상과 음성을 재인코딩하지 않고 MP4 컨테이너만 다시 구성했습니다.
- 파일 끝부분의 **moov** atom을 앞부분으로 이동해 재생 정보를 먼저 읽을 수 있도록 했습니다.
- faststart가 성공하면 최적화된 파일을 반영하고, 실패하면 원본을 보존한 채 업로드 완료로 처리했습니다.
- 업로드 완료 요청 안에서 동기적으로 처리하고, 별도의 작업 큐나 처리 상태를 관리하지 않았습니다.

결과

- 전체 파일 다운로드가 끝나기 전에도 플레이어가 재생을 준비할 수 있는 구조로 개선했습니다.
- 5초 테스트 영상에서 파일 시작 지점부터 **moov** atom 끝까지의 범위가 **약 829KB에서 4.4KB로** 줄었습니다.
- 재인코딩하지 않아 화질 저하와 추가 인코딩 비용을 피했습니다.

배운 점

- DB, 객체 스토리지, HTTP 응답이 정상이어도 문제의 원인이 파일 포맷 내부에 있을 수 있음을 배웠습니다.
- 사용자 체감 지연을 분석할 때는 애플리케이션과 인프라뿐 아니라 실제 파일 구조까지 확인해야 하는 경우도 있다는 점을 경험했습니다.

NEXA — 대화 흐름을 고려해 말하는 Discord AI 멤버

응답 제어

프로젝트 리드(1인)

2026.04 ~ 현재 discord-ai.yeon.world

Kotlin

Spring Boot

JDA

PostgreSQL

Flyway/JPA

WebSocket

Ollama

Docker

운영 범위

운영 도메인 1개

공개 제품 URL과 안전한 공개 기술 문서만 제공합니다.

Discord에서 AI가 모든 메시지에 반응하지 않고, **대화 흐름을 보고 말할지 기다릴지 판단한 뒤 필요한 경우에만 응답**하도록 백엔드 구조를 나눴습니다. 대화 맥락·참여 판단·관계 기록·문장 생성·전송을 각각 분리해 정책을 따로 검증할 수 있게 했습니다.

문제 · 확인 · 결과	
문제	모든 메시지가 하나의 자동응답 흐름에 섞여 AI가 말하거나 기다린 이유를 따로 검증하기 어려움
바꾼 점	대화 맥락·참여 판단·관계 기록·발화·전송을 나누고, 실제 전송 전 SHADOW 단계에서 먼저 검증
확인 가능한 점	프롬프트만 바꾸는 대신 상태·권한·기억·발화 정책을 가진 백엔드 구조로 설명 가능

- 문제
- 명령·멘션·자동 응답이 하나의 흐름에 섞여 있어, AI가 **왜 지금 말했는지 또는 기다렸는지**를 따로 확인하기 어려웠다.
 - 대화에 참여하는 AI는 매 메시지에 답하면 방해가 된다. 언제 개입하고 무엇을 기억하며 어떤 톤으로 답할지 일관된 기준이 필요했다.
 - LLM에게 매번 발화 여부를 묻기만 하면 비용과 지연이 늘고, 판단 근거를 기록하거나 정책을 바꾸기 어렵다.

- 확인한 사실
- 코드와 ADR에서 책임을 대화 맥락, 참여 판단, 관계 기록, 발화, 실제 전송으로 나누고 각 단계의 입력·출력을 분리했다.
 - 실제 메시지를 보내기 전에는 판단 결과만 기록하는 **SHADOW** 단계로 정책을 확인하고, 허용 목록·권한·발화 제한을 통과한 경우에만 **LIVE**로 전송하도록 했다.
 - 메시지 관찰 결과가 관계 상태와 감정을 거쳐 응답 톤으로 이어지도록 연결해, 프롬프트 문구만 바꾸지 않고 상태 변화도 추적할 수 있게 했다.

- 바꾼 점
- 대화 맥락 모듈은 상황만 만들고, 참여 판단 모듈은 무시·대기·반응·발화 중 하나만 선택하며, 발화·전송 모듈은 문장 생성과 전송만 담당하도록 분리.
 - 호명·멘션·답장·대화 연속성은 발화 후보로, 미완성 문장·타인에게 한 말·끝난 대화·AI 자신의 메시지는 대기 또는 무시로 분류.
 - 관계 정보는 친밀도·호감·신뢰·편안함으로 나누고, 한 번의 칭찬처럼 약한 신호가 신뢰를 과도하게 바꾸지 않도록 갱신 규칙을 분리.
 - 정책을 실제 전송 전에 검증하고, 발화 제한·권한·허용 채널을 통과할 때만 활성화.

- 결과
- 대화 관찰·발화 판단·관계 기록·응답 전송을 따로 확인하고, 기능을 단계적으로 켜고 끌 수 있는 구조가 됨.
 - 기능을 한 번에 공개하지 않고 **관찰 → 예측 기록 → 실제 발화** 순서로 올릴 수 있게 됨.
 - 관계와 감정 상태가 응답 톤에 반영되는 경로를 마련해, 운영 중 어떤 상태가 어떤 말투에 영향을 줬는지 추적할 기반을 만들.

- 배운 점
- 대화에 참여하는 AI는 모델 성능만의 문제가 아니라, **상태·기억·발화 타이밍·권한·실행**을 나누어 운영하는 백엔드 설계 문제라는 점을 확인했습니다.
 - 사회적 관계와 신뢰는 단일 점수로 뭉치면 위험하므로, 축을 나누고 보수적인 규칙으로 갱신해야 테스트와 운영이 가능하다는 점을 배웠습니다.

YEON Platform — 여러 서비스를 운영하며 랭킹 API 지연 개선

성능 개선

프로젝트 리더(1인)

2026.04 ~ 현재 yeon.world

Spring Boot

PostgreSQL

QueryDSL

k6

Nginx

cloudflared

Docker

Linux

운영 범위

운영 도메인 23개

공개 제품 URL과 안전한 공개 기술 문서만 제공합니다.

타자연습·플래시카드·커뮤니티·뉴스를 하나의 브랜드와 계정 흐름으로 개발·운영하는 개인 플랫폼입니다. 사용자가 여는 랭킹 화면이 느려지는 문제를 **부하 테스트와 DB 실행 계획**으로 확인하고, 짧게 보관하는 캐시를 적용했습니다.

문제 · 확인 · 결과

문제	사용자가 랭킹을 열 때마다 전체 좋아요를 다시 묶고 정렬해 같은 DB 작업이 반복됨
바꾼 점	5초 TTL 캐시를 두고 좋아요 변경 시 즉시 비워, 반복 조회는 빠르게 하고 변경은 다음 조회에 반영
로컬 측정	동일 50 VU·30초 k6: HTTP 요청/초 431.6 → 56,827, p95 151.9ms → 1.4ms. 운영 처리량 주장이 아님
추가 검증	DB 실행 계획: 기존 집계 46.8ms, 요약 테이블·정렬 인덱스 대안 0.067ms

문제

- 여러 서비스를 한 브랜드로 운영하면서도, 사용자가 어느 서비스에서든 일관된 계정·지원·변경 안내를 받을 수 있어야 했다.
- 게임 좋아요 랭킹은 요청마다 전체 좋아요를 다시 묶고 정렬했다. 사용자가 같은 랭킹 화면을 열 때마다 DB가 같은 집계를 반복해 응답이 느려졌다.

확인한 사실

- 동일한 로컬 환경에서 가상 사용자 50명이 30초 동안 요청하는 부하 테스트 도구 k6를 전후에 같은 조건으로 실행해, 처리량과 p95 지연(느린 5% 요청의 기준 시간)을 비교했다.
- PostgreSQL 실행 계획인 **EXPLAIN ANALYZE** 로 매 요청의 전체 테이블 스캔·집계·정렬을 확인했다. 인덱스가 있어도 매번 결과를 계산하는 구조가 병목이었다.

바꾼 점

- 서비스는 서버도메인으로 분리하고, 계정 흐름·사용법·변경 소식은 허브와 공개 문서로 연결해 사용자가 서비스별 차이를 이해할 수 있게 했다.
- 랭킹 결과를 5초 동안 보관하는 **TTL 캐시**로 바꿨다. 같은 결과를 짧은 시간 안에 다시 요청하면 DB 집계를 반복하지 않고, 좋아요가 바뀌면 캐시를 즉시 비워 다음 요청에서 새 결과를 계산했다.

결과

- 같은 로컬 k6 조건(50 VU, 30초)에서 HTTP 요청 처리량은 **초당 431.6 → 56,827**, p95 지연은 **151.9ms → 1.4ms**였다. 이는 운영 트래픽이 아니라 동일 조건의 로컬 비교 결과다.
- 추가 DB 실험에서 기존 집계는 46.8ms, 요약 테이블과 정렬 인덱스를 쓴 대안은 0.067ms 실행 계획으로 나타났다. 이 값은 캐시 결과와 별개인 쿼리 구조 비교다.
- cloudflared 터널·Nginx·개인 Linux 인프라로 여러 서비스를 공개 URL에서 운영하고 있다.

배운 점

- 여러 독립 서비스를 하나의 계정·도메인·문서 흐름으로 묶어 운영하는 방식을 정리했습니다.
- 게스트 데이터 이관처럼 비회원 상태가 로그인 이후 데이터와 연결되는 흐름을 설계했습니다.

PULL-IT — 오래 걸리는 AI 작업을 사용자 응답에서 분리

비동기 처리

팀장 (5인)

2025.09 ~ 2025.11 pull.it.kr

Spring Boot

RabbitMQ

Gemini

SSE

OAuth2

Docker

GitHub Actions

운영 범위

운영 도메인 10개

공개 제품 URL과 안전한 공개 기술 문서만 제공합니다.

PDF에서 AI가 문제를 만드는 동안 사용자가 응답을 기다리지 않도록, 요청을 받는 서버와 오래 걸리는 작업을 처리하는 서버를 분리했습니다. 두 서버가 메시지 큐(RabbitMQ)로 협력할 때 생기는 **작업 순서·실패·중복·완료 알림**을 각각 설계했습니다.

문제 · 확인 · 결과

문제	AI 생성이 오래 걸려 사용자가 기다리고, 서버 분리 뒤에는 커밋 순서·실패·알림 상태를 따로 다뤄야 함
바꾼 점	커밋 후 메시지 발행, Worker→Main 완료 큐, 생성/알림 상태 분리, 단일 트랜잭션 저장
해당 경로 측정	문제 30개 저장: 트랜잭션 30 → 1, prepared statement 준비 횟수 120 → 62, 저장 지연 약 69% 감소

문제

- AI 문제 생성은 수 초가 걸린다. 요청 처리 안에서 끝까지 기다리게 하면 사용자는 화면 응답을 오래 기다리고, 서버도 다음 요청을 처리하기 어려워진다.
- 작업을 두 서버로 나누면 사용자는 빨리 응답받지만, DB 저장 전 메시지가 전달되거나 작업은 성공했는데 알림만 실패하는 새 문제가 생긴다.

확인한 사실

- Main 서버는 요청을 받아 작업 정보를 DB에 저장하고 접수 응답을 보낸다. Worker 서버는 이후 메시지를 받아 AI 생성과 문제 저장을 수행한다.
- 문제 30개를 저장하는 실제 MariaDB 경로에서 Hibernate Statistics를 수집해, 트랜잭션 수·DB에 준비한 SQL 문 수(prepared statement count)·저장 지연을 변경 전후 비교했다.

바꾼 점

- Main은 DB 커밋이 끝난 뒤에만 작업 메시지를 보냈다(`@TransactionalEventListener(AFTER_COMMIT)`). 그래서 Worker가 아직 저장되지 않은 작업을 읽는 순서 문제를 막았다.
- 일시적인 AI 호출 오류와 다시 시도해도 해결되지 않는 오류를 나누고, 재시도 횟수를 넘으면 실패 상태를 남겼다. 같은 메시지가 계속 실패하는 경우가 큐를 막지 않게 했다.
- Worker가 완료 사실을 두 번째 큐로 Main에 보내고, Main만 SSE로 사용자에게 완료를 알렸다. 생성 성공 상태와 알림 전송 상태를 분리해 알림 실패가 완성된 문제집을 실패로 되돌리지 않게 했다.
- 문제 저장은 한 문제씩 여러 번 DB에 보내는 대신, 한 트랜잭션에서 묶어 저장하도록 바꿨다. 중복 검증 조회를 줄이고 저장 단위를 명확히 했다.

결과

- 사용자는 문제 생성이 끝날 때까지 기다리지 않고 요청 접수 응답을 받으며, 완료되면 별도 알림을 받는 흐름이 됐다.
- 실제 MariaDB·Hibernate Statistics 측정에서 문제 30개 저장은 **트랜잭션 30→1, prepared statement 준비 횟수 120→62, 저장 지연 약 69% 감소**였다. 이 값은 해당 저장 경로의 측정값이다.
- ID 생성 전략이 배치 저장을 막을 수 있음을 같은 옵션의 통제 실험으로 확인했다. IDENTITY는 30회, SEQUENCE는 2회 실행으로 관찰돼, 위험 대비 효과가 큰 저장 방식만 적용했다.

배운 점

- 서버를 분리하면 응답 경로는 가벼워지지만, 메시지 순서·실패 처리·중복 방지 기준을 별도로 설계해야 한다는 점을 확인했습니다.
- 작업 완료 상태와 완료 알림 상태를 분리하고, 설정값이 실제 배치 동작으로 이어지는지 지표로 확인했습니다.

ZERO-ONE — 비슷한 조건의 학습자를 자동으로 연결

운영 자동화

백엔드 (8인 팀)

2025.01 ~ 2026.03

zeroone.it.kr

Spring Boot

Spring Batch

Scheduler

JPA

MySQL

운영 범위

운영 도메인 16개

공개 제품 URL과 안전한 공개 기술 문서만 제공합니다.

개발자 1:1 CS 스터디에서 사용자가 관심 주제와 가능한 시간대가 비슷한 사람을 만나도록 자동 매칭을 만들었습니다. 선호 정보를 수치로 바꾸어 유사한 사람을 묶고, 같은 조합이 반복될 가능성은 가중치 기반 무작위 선택으로 낮췄습니다.

문제 · 확인 · 결과	
문제	비슷한 사람을 만나되 같은 조합이 자주 반복되지 않고, 운영자는 수동 매칭을 줄여야 함
바꾼 점	선호 정보를 벡터로 만들고 K-Means로 유사 그룹을 만든 뒤, 그룹 안에서 가중치 기반 무작위 선택으로 반복 가능성을 낮춤
범위	매칭·스터디 공간·참여자·출석 초기화 자동화는 구현함. 성능 수치는 별도 실험 전까지 주장하지 않음

- 문제

 - 사용자는 관심 주제·가능 시간대·학습 성향이 비슷한 사람과 만나고 싶지만, 같은 상대 조합이 계속 반복되는 경험은 피하고 싶다.
 - 운영자는 매 주기마다 사람을 수동으로 묶지 않아도, 매칭부터 스터디 공간·참여자·출석 초기화까지 이어지길 원했다.
- 확인한 사실

 - 선호 주제·가능 시간대·학습 성향·기술 스택을 매칭에 쓸 수 있는 숫자 묶음으로 정리했다. 이렇게 만든 숫자 묶음을 **벡터**라고 한다.
 - 가까운 벡터를 같은 그룹에 모으는 **K-Means** 알고리즘을 적용하고, 그룹 안에서는 이전 조합을 고려한 가중치 기반 무작위 선택을 적용했다. 매번 새 조합을 보장하는 방식은 아니다.
- 바꾼 점

 - 매칭 조건마다 중요도를 다르게 두었다. 예를 들어 어떤 스터디는 주제와 시간대를 더 크게 보고, 다른 스터디는 성실도를 더 크게 반영할 수 있게 했다.
 - Spring Batch와 Scheduler로 주말마다 **매칭 → 스터디 공간 생성 → 참여자 등록 → 출석 초기화**가 이어지도록 자동화했다.
- 결과

 - 주기별 매칭이 자동 실행되고, 매칭 결과부터 출석 기록까지 한 흐름에서 일관되게 처리되도록 만들었다.
 - 11개 도메인 테이블의 관계를 정의해, 사람을 고르는 일과 이후 스터디 운영 준비가 끊기지 않게 연결했다.
 - 인덱스·배치·스케줄러의 성능 수치는 아직 별도 실험으로 측정하지 않아 이 포트폴리오에서 성능 개선 수치로 주장하지 않았다. 현재는 AI 지원 개발 학습 플랫폼으로 운영 방향을 전환했다.
- 배운 점

 - 추천/매칭 요구사항을 벡터·거리·확률 모델로 구현했습니다.
 - 배치·스케줄러 기반 자동화와 복합 도메인 트랜잭션을 설계했습니다.